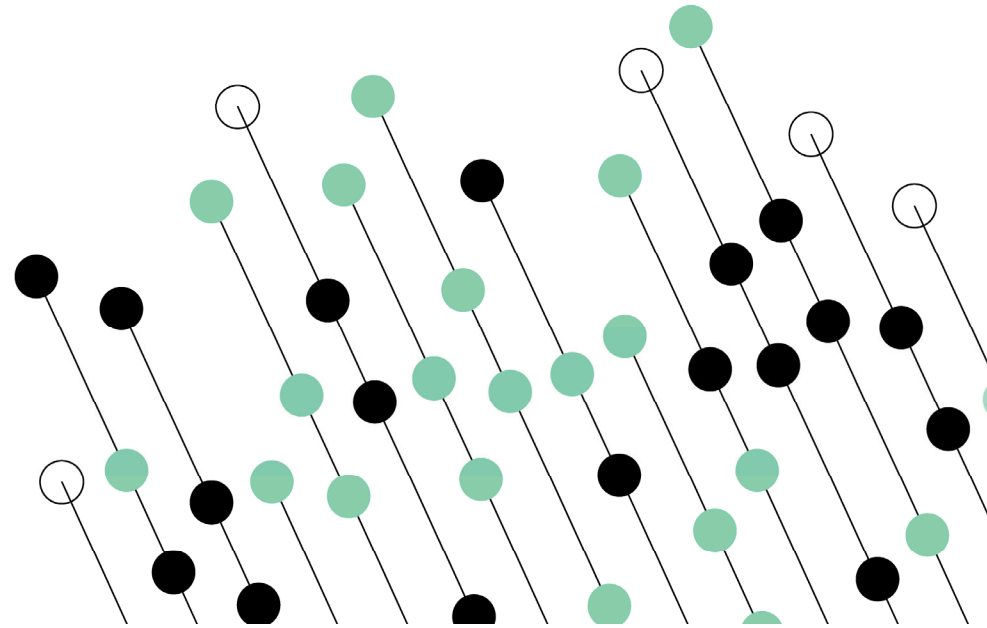


Collections and generics



Contents



Objectives

- Compare functionality offered by arrays and collections.
- Understand generic concepts, use generic types and syntax.



Contents

- Recap arrays, introduce collection classes.
- Generic concepts:
 - Collections framework.
 - Generic classes.



Hands-on labs

Arrays vs. collection classes

- **Limitations of arrays**
 - Fixed size; can't append, insert, or delete.
 - No built-in method to reject duplicates.
 - Must watch out for [ArrayIndexOutOfBoundsException](#).
 - But are type-safe — an **int** array can only contain **ints**.
- **Java offers collection classes**
 - Queue, Stack, List, Set, Map, Dictionary, SortedList.
 - Know their capacity and count.
 - Support append, insertions / deletions / searching.
 - Generic version of these are type-safe.



Collection classes before Java 5 (2004)

- Classic `ArrayList` could only hold a collection of `Object` type.

```
ArrayList myList = new ArrayList();  
  
myList.add(123);  
  
myList.add("Bob");
```

Java's `get()` method returns an `Object` at an index

```
int id = myList.get(0);
```

✗ needs casting. `get()` returns an `Object`

```
int id = (int)myList.get(0);
```

✓ Cast is valid, but it takes time to cast `Object`

```
int id = (int)myList.get(1);
```

Compiles but crashes during runtime

Generic collection classes

- Can hold a collection of a specific type and always returns the expected type.

```
ArrayList<Integer> myList = new ArrayList<>();
```

```
myList.add(123);
```

✓ can add an integer

```
myList.add(123.5);
```

✗ double is not allowed

```
myList.add("Bob");
```

✗ Only integers

```
int k = myList.get(0);
```

✓ no casting is required

import java.util.ArrayList

```
ArrayList<Double> salaries = new ArrayList<>();
```

```
salaries.add(123.5);
```

✓ can add a double

```
double x = salaries.get(0);
```

✓ no casting is required

Iterating through a generic ArrayList

```
ArrayList<String> friends = new ArrayList<>();  
friends.add("Tom");  
friends.add("Sue");  
friends.add("Sanjeev");
```

add() expects a String

```
for (String name : friends)  
    System.out.printf(name);
```

enumerable

```
for(int i = 0; i < friends.size(); i++)  
    System.out.printf( friends.get(i) );
```

Note the size()

```
friends.set(1, "Susan");
```

Susan replaces Sue

Why use **List** instead of **ArrayList** as reference type?

When you declare a variable as **List** you're programming to an interface (List).

```
List<String> names = new ArrayList<>();
```

vs

```
ArrayList<String> names = new ArrayList<>();
```

It provides the flexibility to change implementations.

You can decide to use a **LinkedList** instead:

```
names = new LinkedList<>();
```

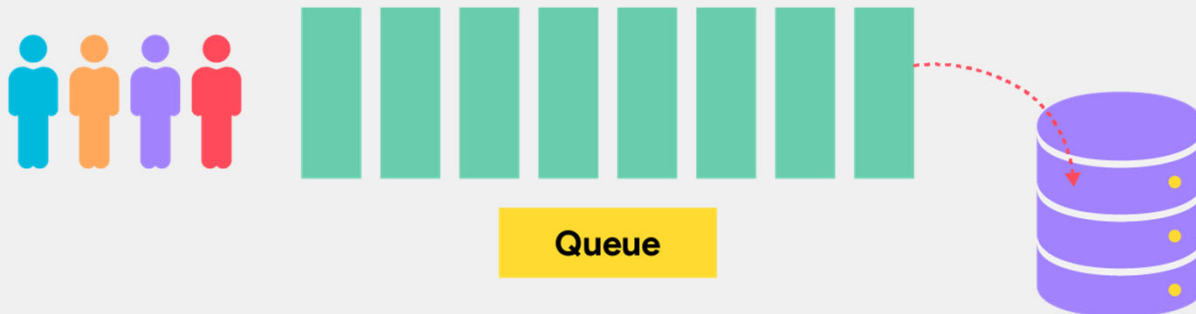
If you had declared it as **ArrayList**, you must modify all occurrences of **ArrayList**.

Java generic collections framework types

- In package `java.util`

```
public class ArrayList<T> { ... }  
public class LinkedList<T> { ... }  
public class Queue<T> { ... }  
  
public class HashSet<T> { ... }  
public class TreeSet<T> { ... } // Sorted  
  
public class HashMap <K,V> { ... }  
public class TreeMap <K,V> { ... }
```

Multiple type
parameters allowed



Java: Queue FIFO

```
Queue<String> printJobs = new LinkedList<>();
printJobs.add("Invoice.pdf");
printJobs.add("Report.docx");

while (!printJobs.isEmpty()) {
    String job = printJobs.poll();
    System.out.println("Printing: " + job);
}
System.out.println("Q empty? " + printJobs.isEmpty());
```

```
Printing: Invoice.pdf
Printing: Report.docx
Is the Q empty? true
```

```
Queue<Car> carsQ = new LinkedList<>();

carsQ.add(new Car("Ford"));
carsQ.add(new Car("Honda"));

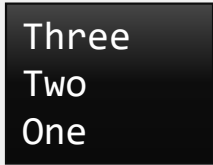
while (!carsQ.isEmpty()) {
    Car car = carsQ.poll();
    System.out.println(car.getModel());
}
```



```
Ford
Honda
```

Java: Stack — LIFO

```
Deque<String> stack = new ArrayDeque<>();  
  
stack.push("One");  
stack.push("Two");  
stack.push("Three");  
  
System.out.println(stack.pop());  
System.out.println(stack.pop());  
System.out.println(stack.pop());
```

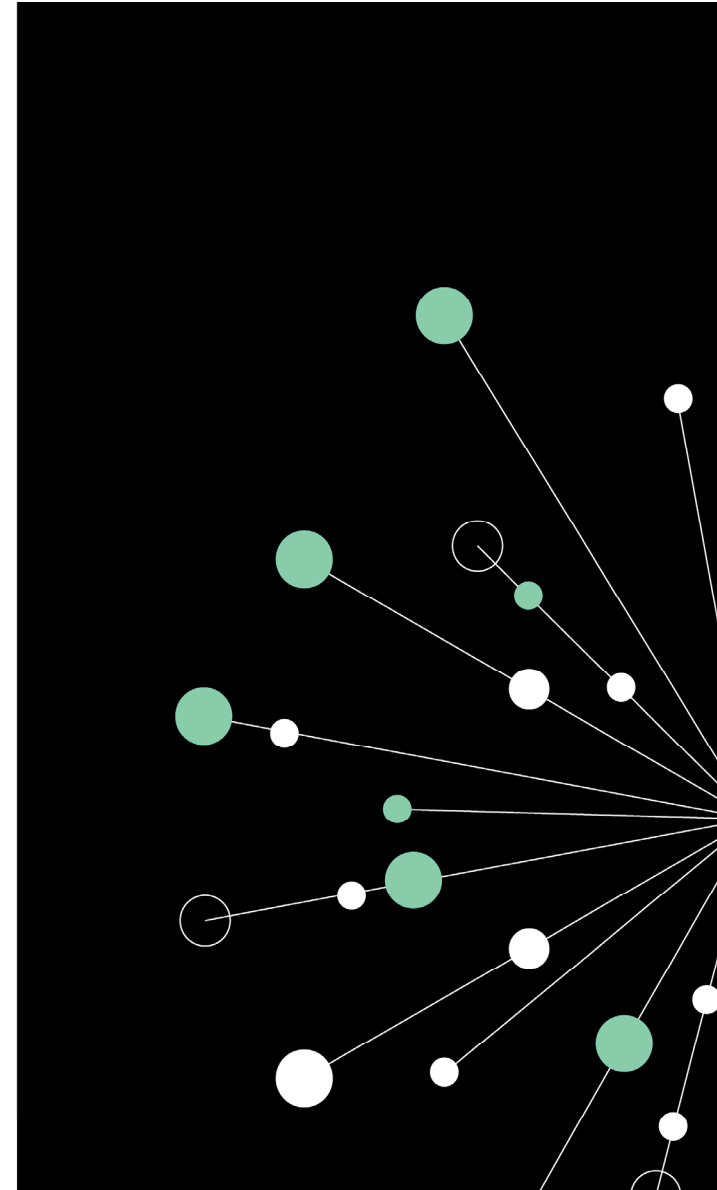


Three
Two
One

Lab

- Part 1 — Using lists
- Part 2 — Using queues

QA



Key value pairs

Java `HashMap` class

- It stores and manages any type of item.
- Each item has a unique key.
- You can get an element using its key.
- The key can be of any type, not just integers.
- It's more efficient than an `ArrayList`.

Key	Value
"AB102"	200.5
"JF786"	984.9
"KE900"	765.75



Java: Usage of HashMap of key / value pairs

```
HashMap<String, Car> hm = new HashMap<>();  
  
hm.put("Sam", new Car("Ford"));  
hm.put("Joe", new Car("BMW"));  
  
Car car = hm.get("Sam");  
  
System.out.println(car.getModel());
```

Ford

Using the same key replaces the item

```
if(!hm.containsKey("Bob")) {  
    hm.put("Bob", new Car("Ferrari"));  
}
```

Searching and
replacing an item

Java: Usage of HashMap of key / value pairs

```
HashMap<String, Car> hm = new HashMap<>();
```

```
hm.put("Sam", new Car("Ford"));
```

```
hm.put("Joe", new Car("BMW"));
```

```
Joe drives a BMW  
Sam drives a Ford
```

```
for (String key : hm.keySet()) {  
    System.out.printf("%s drives a %s\n", key, hm.get(key).getModel());  
}
```

```
for (Car car : hm.values()) {  
    System.out.println(car.getModel());  
}
```

```
BMW  
Ford
```

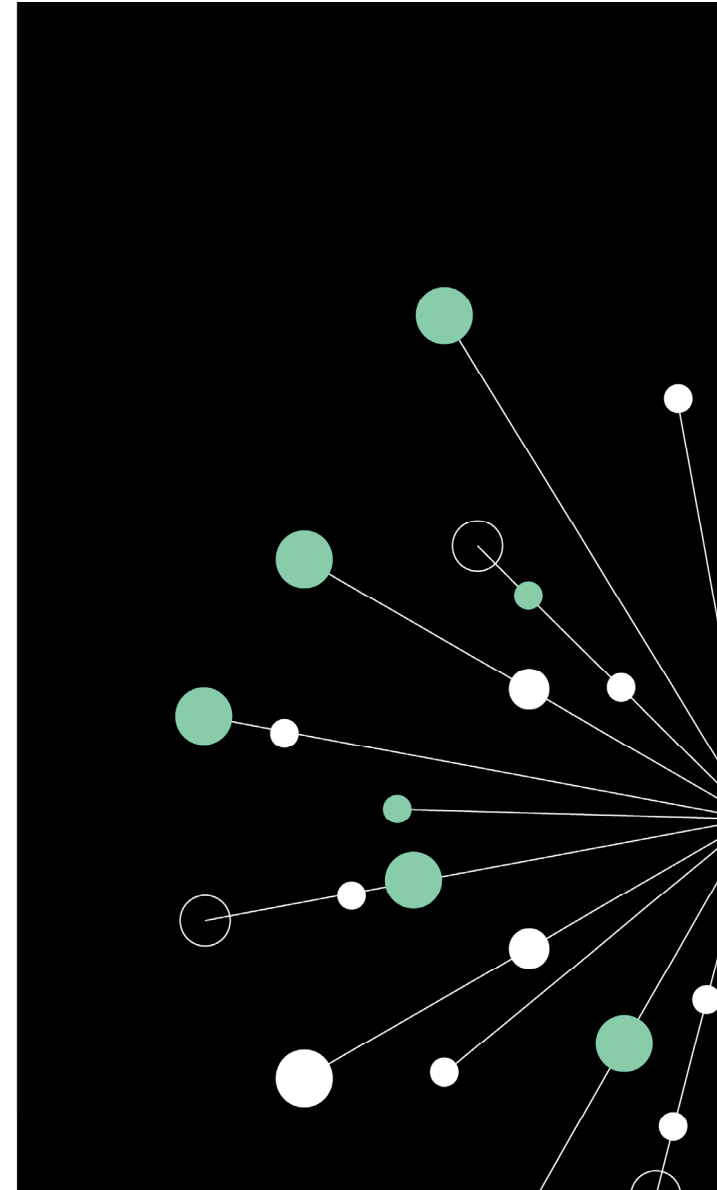
Review

- **Java 5.0 (2004) introduced generic types**
 - Improve performance and type safety
- `ArrayList<E>` & `HashMap<TKey, TValue>`
- Deque and Queue behaviour
- `java.util` package
- **Maps** are key value pairs (like a dictionary)
 - Implemented by `HashMap` and `TreeMap`(sorted)



Lab

- Zoo animals
 - Using `HashMap<K, V>`



LinkedList

- **LinkedList is a linear data structure where elements are linked together.**
 - Each element contains data and a reference (pointer) to the next node.
- Unlike arrays, elements are not stored in contiguous memory locations.
- **Types**
 - **Singly Linked List** — Each node points to the next node.
 - **Doubly Linked List** — Nodes have pointers to both next and previous nodes.
 - **Circular Linked List** — The last node connects back to the first node.



LinkedList code

```
LinkedList<String> list = new LinkedList<>();  
  
list.add("Alice");  
list.add("Bob");  
list.addFirst("Zara");           // Add at the beginning  
list.addLast("Charlie");         // Add at the end
```

Adding elements

```
list.removeFirst();              // Removes "Zara"  
list.removeLast();              // Removes "Charlie"
```

Removing elements

```
System.out.println("First Element: " + list.getFirst());  
System.out.println("Last Element: " + list.getLast());
```

Access elements

```
for (String name : list) {  
    System.out.println(name);  
}
```